

リモセン応用セミナー WEBGIS実装講座

衛星データ解析技術研究会 技術セミナー（応用編）

第1回 タイル配信①

データ構造とパース

2026年6月19日（金）

到達目標：タイル配信のエコシステムについて説明できるようになる。/ベクター形式、ラスター形式の地図配信について理解する。

タイムライン

時間	項目
0:00–0:05	ガイダンス
0:05–0:20	WebGIS タイル配信エコシステムの概論
0:20–0:35	タイルピラミッド・座標系・リクエストの仕組み
0:35–0:50	ラスタータイルとベクタータイルの違い（MVT / PMTiles）
0:50–1:00	— 中休み —
1:00–1:25	タイルサーバー比較（GeoServer / Martin / SaaS）
1:25–1:45	GIS レンダリングエンジンのタイル処理を読む
1:45–2:20	ハンズオン：tippecanoe で PMTiles 生成 → MapLibre 確認
2:20–2:30	まとめ・次回への接続

ガイダンス

本日の到達目標

- タイルピラミッドの仕組みを説明できる
- MVT (protobuf) と PMTiles の関係を整理できる
- `tippecanoe` で PMTiles を生成し、MapLibre で表示できた

「外部サービスをブラックボックスとして使わず、データの流れを自分の言葉で整理できる」状態を目標とする

エコシステム概論

WebGIS タイル配信のスタック

データ生成	変換・生成	配信	クライアント
QGIS	GDAL / ogr2ogr	Martin (Rust)	MapLibre GL JS
PostGIS	tippecanoe	Nginx + PMTiles	OpenLayers
衛星画像・DEM	rio-cogeo	MapTiler Cloud	Leaflet

本講義のスコープ：変換・生成 → 配信 → クライアント の流れを自分で組む

OSGeo / FOSS4G コミュニティが上記ツール群の多くを支えている。MapLibre は Mapbox の 2021 年フォーク。

タイルピラミッド

ズームレベルとタイル分割

ズームレベル `z` において、世界地図は $2^z \times 2^z$ 枚に分割される。

ズーム	タイル数	スケール感
Z=0	1枚	世界全体
Z=1	4枚	
Z=2	16枚	
Z=10	約 100万枚	市区町村レベル
Z=17	約 170億枚	農地区画・建物レベル

ズームが1上がるごとにタイル数は4倍。高ズームの全タイル生成はストレージが膨大になるため、tippecanoe では生成範囲を `-Z / -z` で絞る。

タイルのアドレス指定

`https://example.com/tiles/{z}/{x}/{y}.pbf`

例：東京駅付近・ズーム17

`https://example.com/tiles/17/116561/51781.pbf`

- `x` : 西から数えたタイル番号 ($0 \sim 2^z - 1$)
- `y` : 北から数えたタイル番号 (XYZ 規格の場合)

規格	y 軸の向き	備考
XYZ (Slippy Map)	北が 0	MapLibre・Leaflet のデフォルト
TMS	南が 0 (反転)	<code>gdal2tiles</code> のデフォルト出力がこれ
WMTS	OGC 標準	GISサーバー系との互換

gdal2tiles で生成したタイルは TMS 形式 → MapLibre へ渡す際に y 軸の変換が必要
第1回：タイル配信①：データ構造とパース

座標系の整理

EPSG:4326 / 3857 / JGD2011

コード	名称	用途
EPSG:4326	WGS84	緯度・経度の10進数表現。GeoJSON・GPS データのデフォルト。
EPSG:3857	Web Mercator	タイルグリッド・地図表示のデファクト標準。メートル単位の平面座標。
EPSG:6668	JGD2011	日本の公的データ（農水省筆ポリゴン等）の配布座標系。そのままでは使えない。

```
# JGD2011 → WGS84 変換
ogr2ogr -f GeoJSON output.geojson input.shp -t_srs EPSG:4326
```

ハンズオンの Step 1 でこの変換を実施する。

タイルの種類

ラスタータイルとベクタータイル（MVT）

	ラスタータイル	ベクタータイル（MVT）
形式	PNG / JPEG 画像	protobuf（バイナリ）
レンダリング	サーバー側で事前生成	クライアント（GPU）側
スタイル変更	不可（再生成が必要）	リアルタイムに変更可能
滑らかさ	ズーム段差あり	スムーズ
主な用途	衛星画像・Terrain RGB	地物・道路・境界

本講義はベクタータイル（MVT / PMTiles）を主軸とする。ラスタータイルは第3・4回（DEM）で扱う。

データ構造

MVT (Mapbox Vector Tile) の構造

```
.pbf ファイル (バイナリ)  
└─ layer: "buildings"  
  │─ feature { geometry: Polygon, props: {height: 20} }  
  │─ feature { geometry: Polygon, props: {height: 35} }  
  └─ ...  
└─ layer: "roads"  
  │─ feature { geometry: LineString, props: {name: "国道2号"} }  
  └─ ...
```

- 座標は **0～4096 の整数**（タイル内相対値）でエンコード → 軽量
- 実座標への変換はクライアント側で行う
- 仕様：[mapbox/vector-tile-spec](https://mapbox.com/docs/vector-tiles/)

データ構造

PMTiles とは

複数の MVT タイルを **1ファイルにアーカイブ**した形式。

```
fude.pmtiles (単一ファイル)
├─ ヘッダー (インデックス: タイル座標 → バイト位置の対応表)
├─ z=10, x=..., y=... のタイルデータ
├─ z=11, x=..., y=... のタイルデータ
└─ ... (全ズームレベルのタイルが連続格納)
```

HTTP Range Request (206 Partial Content) を使い、ブラウザが必要なタイルのバイト範囲だけを取得する。

- タイルサーバーを立てなくても **Nginx / S3 / CDN** で配信可能
- MBTiles (SQLite) との違いはアーカイブ方式のみ

タイルサーバー比較

GeoServer / Martin / tileserver-gl / SaaS

サーバー	実装	PMTiles	起動コスト	主な用途
GeoServer	Java / WAR	直接配信不可	重い（JVM）	OGC 標準準拠が必要なエンタープライズ連携
Martin	Rust / 単一バイナリ	✓	軽い（即起動）	PostGIS / MBTiles / PMTiles を統一配信
tileserver-gl	Node.js	✓	中程度	ラスタータイルの動的生成が必要な場合
MapTiler / Mapbox	SaaS	プラン依存	最小	プロトタイピング

本講義では Martin + PMTiles を軸にする。GeoServer は「起動して URL を確認する」デモに留める。SaaS はリクエスト数超過で課金・ロックインリスクに注意。

Martin の起動と MapLibre からの接続

```
# インストール
brew install martin

# PMTiles を配信
martin fude.pmtiles
# → http://localhost:3000/fude/{z}/{x}/{y}
```

```
// MapLibre からの接続
map.addSource('fude', {
  type: 'vector',
  tiles: ['http://localhost:3000/fude/{z}/{x}/{y}']
});
```

確認ポイント： Network タブで `/z/x/y` へのリクエストがズームに連動して発生することを確認。GeoServer の WMTS URL（長いクエリパラメータ）と比較する。

MapLibre GL JS がタイルを処理する流れ

- 1. スタイル JSON 読み込み → `source` の URL を確認
- 2. viewport + zoom から必要タイル座標 (z/x/y) を計算
- 3. タイル URL に Fetch リクエスト
- 4. 受け取った protobuf (MVT) をデコード
- 5. WebGL にジオメトリを転送 → GPU でレンダリング

処理	内部ライブラリ
GeoJSON → タイル変換	<code>geojson-vt</code>
protobuf デコード	<code>@mapbox/vector-tile</code> + <code>pbf</code>
レンダリング	WebGL

GIS レンダリングエンジン

GeoJSON source と vector source の処理コスト

GeoJSON source を使う場合、MapLibre は受け取った GeoJSON を内部で `geojson-vt` によってタイルに変換してからレンダリングする。

→ クライアントがタイル変換のコストを負担する

フィーチャー数が **10万オーダーを超えると**、この変換処理がボトルネックになる。

vector source（PMTiles / MVT）を使う場合、タイル変換済みのバイナリを直接デコードして GPU に渡す。

→ `geojson-vt` の変換コストは発生しない

実例

大量ポリゴンが描画されない問題：地番図を例に

	内容
状況	10万オーダーの地番ポリゴンを GeoJSON から直接読み込み
症状	レンタルサーバーでデータは到達しているが描画されない
初期仮説	ネットワーク転送が原因？
切り分け	<code>curl</code> → GeoJSON 到達確認。DevTools Memory → メモリ展開済みと確認
原因	MapLibre 内部の <code>geojson-vt</code> 変換処理が CPU ボトルネック
解決	GeoJSON → PMTiles (tippecanoe) に変換 → 解消

「ネットワークが原因」という初期仮説が外れた点が重要。DevTools の Memory タブでの切り分けが有効。

ツールのインストール

GDAL / ogr2ogr

```
# macOS
brew install gdal

# Linux (Ubuntu/Debian)
sudo apt install gdal-bin

# Windows → OSGeo4W インストーラー（QGIS に同梱されていれば不要）
# https://trac.osgeo.org/osgeo4w/
```

tippecanoe

```
# macOS
brew install tippecanoe

# Linux (Ubuntu/Debian)
sudo apt install tippecanoe
```


ハンズオン

tippecanoe で PMTiles を生成して MapLibre で表示する

使用データ：

農林水産省「農業用地利用状況調査（筆ポリゴン）」

<https://open.fude.maff.go.jp/> → 山口県の Shapefile をDL

1. Step 1：ogr2ogr で座標変換（JGD2011 → WGS84）
2. Step 2：tippecanoe で PMTiles 生成
3. Step 3：MapLibre で表示・確認

ogr2ogr で座標変換

```
ogr2ogr \  
-f GeoJSON fude.geojson \  
ダウンロードした.shp \  
-t_srs EPSG:4326
```

変換後の確認：

```
# macOS / Linux  
ls -lh fude.geojson  
wc -l fude.geojson  
  
# Windows (PowerShell)  
Get-Item fude.geojson | Select-Object Length
```

- 筆ポリゴンは JGD2011 (EPSG:6668) 配布 → EPSG:4326 に変換しないと tippecanoe に渡せない
- Shapefile が複数ファイル (.shp / .dbf / .prj / .shx) で構成されていることを確認

tippecanoe で PMTiles 生成

```
# macOS / Linux (Windows は WSL 内で実行)
tippecanoe \
  -o fude.pmtiles \
  --force \
  -z 17 -Z 10 \
  --drop-densest-as-needed \
  -l fude \
  fude.geojson
```

オプション	意味
-z 17	最大ズームレベル（農地区画の精度に合わせる）
-Z 10	最小ズームレベル（これより引くとタイルが存在しない）
--drop-densest-as-needed	タイルサイズ上限（500KB）超過時にフィーチャーを自動間引き
-l fude	source-layer 名（MapLibre 側で参照する）

ls -lh fude.pmtiles # GeoJSON の数分の1～数十分の1程度になる

MapLibre で PMTiles を表示する (1/2)

```
// PMTiles プロトコルを登録
const protocol = new pmtiles.Protocol();
maplibregl.addProtocol('pmtiles', protocol.tile);

const map = new maplibregl.Map({
  container: 'map',
  style: {
    version: 8,
    sources: {
      osm: {
        type: 'raster',
        tiles: ['https://tile.openstreetmap.org/{z}/{x}/{y}.png'],
        tileSize: 256
      },
      fude: {
        type: 'vector',
        url: 'pmtiles:///fude.pmtiles'
      }
    }
  }
});
```

MapLibre で PMTiles を表示する (2/2)

```
layers: [  
  { id: 'osm', type: 'raster', source: 'osm' },  
  {  
    id: 'fude-fill', type: 'fill', source: 'fude',  
    'source-layer': 'fude', // tippecanoe の -l に一致させる  
    paint: { 'fill-color': '#6ab04c', 'fill-opacity': 0.5 }  
  }  
]  
},  
center: [131.4, 34.1], // 山口県付近  
zoom: 12  
});
```

```
# macOS / Linux  
python3 -m http.server 8000
```

第1回: スライド配信 (PowerShell と コマンドプロンプト)
python -m http.server 8000

ハンズオン — 比較

GeoJSON 直読み vs PMTiles

比較項目	GeoJSON 直読み	PMTiles
初期ロード	全量転送・メモリ展開	表示範囲のタイルのみ取得
MapLibre 処理	<code>geojson-vt</code> で全フィーチャを変換	タイル単位でインクリメンタルにデコード
ファイルサイズ	数百MB（圧縮前）	tippecanoe が自動間引き・圧縮
ホスティング	単ファイルだが転送量大	Range Request で部分取得
タイルサーバー	不要	不要（静的配信で動作）

確認： DevTools の Network タブで PMTiles への Range Request を観察する。GeoJSON source に切り替えて差を体感する。

まとめ

本日の到達確認

- [] タイルピラミッドの仕組みを説明できる
- [] XYZ / TMS / WMTS の y 軸の違いを区別できる
- [] MVT (protobuf) と PMTiles の関係を説明できる
- [] `tippecanoe` で PMTiles を生成できた
- [] MapLibre で `vector` source として表示できた
- [] GeoJSON source と vector source の処理コストの違いを説明できる

第2回（6/26）予告

タイル配信② 自前配信とカートグラフィック

キーワード： Nginx ・ PMTiles 静的配信 ・ CORS ・ Maputnik ・ MapLibre Style Spec ・ OpenStreetMap

- Nginx で PMTiles を配信する（タイルサーバー不要）
- Maputnik でスタイルを GUI 編集 → `style.json` エクスポート
- 同一オリジンポリシーと CORS の関係を整理する
- デモ①：ラズパイ上のオフライン地図サーバーに接続する
- OSM へのデータ還元と現地調査ツールとしての活用
- デモ②：地理院地図ベクタータイルのスタイルカスタマイズ

第2回は現地開催です。前回生成した PMTiles ファイルを持参または共有してください。